

实验三 载波传输

班级：无 32

学号：2023010504

姓名：张乐

日期：2025 年 11 月 26 日

一、 实验背景

实验目的：

1. 通过 Matlab 仿真，熟悉误符号率和误比特率的概念。
2. 通过对矩形包络下的 BPSK 与 4PAM 调制的载波传输进行仿真，深入理解载波传输的收发结构及差错概率。
3. 通过对矩形包络下的 BPSK 与 4PAM 调制的载波传输进行仿真，理解两种不同的匹配滤波方法。
4. 通过对矩形包络下的 BPSK 与 4PAM 调制的载波传输进行仿真，熟练功率谱密度的计算。

Matlab 版本：2023b。

二、 实验步骤

分别在 BPSK 和 4PAM 两种调制方式下：

- (1) 生成 $N = 10^4$ 个随机符号。
- (2) 基带调制利用矩形脉冲成形， $T = 0.01s$ ，采样间隔 $\delta_t = 10^{-4}s$ 。
- (3) 载波采用 $\sqrt{2}\cos 2\pi f_c t$ ， $f_c = 500Hz$ ，生成对应的载波波形，画出前 10 个符号的基带调制波形和载波调制波形。
- (4) 生成高斯噪声，与发送信号相加，得到接收信号，画出前 10 个符号的接收波形。
- (5) 采用两种匹配滤波方式对接收信号进行处理。
方式一：接收信号直接与 $g(t) = \sqrt{2/T}\cos 2\pi f_c t$ 卷积。
方式二：接收信号先乘以 $\sqrt{2}\cos 2\pi f_c t$ 下变频，再与矩形脉冲 $p(t) = \sqrt{1/T}$ 卷积。
画出前 5 个符号的匹配滤波输出波形（采样前），并在图中明确标出最佳采样时刻。
- (6) 比较发送信号和接收信号，统计误符号率、误比特率。把 BPSK 和 4PAM 的误码曲线画一张图上对比，观察 10^{-3} 误比特率所需的 E_b/n_0 有多少 dB 差距，并结合理论公式做解释。（误码曲线纵坐标用对数，信噪比用 dB，SNR 范围要覆盖误码率从 $10^{-4} \sim 10^{-1}$ ）
- (7) 在最佳采样时刻分别引入 1~2 个采样点的定时偏差，对比两种接收方法的 BPSK 误码率曲线。通过改变噪声功率谱密度，绘制误比特率（BER）与 E_b/n_0 以及误符号率（SER）与 E_s/n_0 的关系曲线，并将实验结果与理论误比特率和误符号率曲线进行比较分析。
- (8) 利用窗函数法画出载波信号的功率谱密度，并与理论值进行比较。

三、 BPSK 调制

1. 理论推导和计算

在 BPSK 调制下，误符号率和误码率分别为：

$$P_s = Q\left(\sqrt{\frac{E_s}{n_0/2}}\right)$$

$$P_b = Q\left(\sqrt{\frac{E_b}{n_0/2}}\right)$$

BPSK 调制后，载波信号的功率谱密度为：

$$S_x(f) = \frac{A^2 T}{2} \left[\left(\text{sinc}((f - f_0)T) \right)^2 + \left(\text{sinc}((f + f_0)T) \right)^2 \right]$$

2. 代码实现

```
% 通信与网络 实验 3 载波传输
%BPSK
clearvars;
close all;
clc;

% 符号个数
N = 1e5;

% 符号能量
Es = 1;

% 脉冲成形波形参数
T = 0.01; % 周期
A = sqrt(Es/T); % 幅度

%载波参数
f_c=500; %载波频率

% 最小时间间隔
delta_t = 1e-4;

%仿真时间
t = (delta_t:delta_t:N*T);

%每个符号周期采样次数
```

```
num=T/delta_t;

% 成形脉冲
p_t = A * ones(num,1);

%初始化接收符号、判决符号
y=zeros(1,N);
x_hat=zeros(1,N);

%基带脉冲成形
x0_t=zeros(1,N*num);

%随机生成符号序列
bit_data = randi([0 1], 1, N);
% 转化为调制后的电平序列 (mod_data)
mod_data = my_gray_map_real_M2(bit_data);

%脉冲成形
for cnt = 1:N
    x0_t(num*(cnt-1)+1:num*cnt) = mod_data(cnt)*p_t;
end

%BPSK 调制
x_t = x0_t.*sqrt(2).*cos(2*pi*f_c*t);

%% ===== Eb/N0 扫描参数 =====
EbN0_dB = 2:2:8; % 不同信噪比(dB)
EbN0 = 10.^(EbN0_dB/10); % 线性比值
EsN0 = EbN0;
EsN0_dB= EbN0_dB;
Eb = Es; % BPSK 情况下满足
N0_list = Eb ./ EbN0 ;

% 虽不是 TODO 范围，但做了修改，改成二维向量匹配后面的双重循环
BER_sim1 = zeros(length(EbN0_dB),3); % 方法 1 仿真 BER
BER_sim2 = zeros(length(EbN0_dB),3); % 方法 2 仿真 BER

% ===== 匹配滤波器定义 =====
% 方法 1: 带通匹配滤波器
h1_t = sqrt(2/T) * cos(2*pi*f_c*(0:delta_t:T-delta_t));
% 方法 2: 基带匹配滤波器
h2_t = (1/sqrt(T)) * ones(1, num);

%最佳采样时刻
t_sample = T: T: N*T;
```

```
idx_sample = round(t_sample/delta_t);

%% 不加噪声观察匹配滤波
y_t = x_t;

% % 匹配滤波输出
%方法 1
mf_out1 = conv_no_delay(y_t, fliplr(h1_t), delta_t);

%方法 2
y_base = y_t.*sqrt(2).*cos(2*pi*f_c*t);
mf_out2 = conv_no_delay(y_base, fliplr(h2_t), delta_t);

% 前 5 个符号输出波形
figure;
subplot(2,1,1)
plot(t(1:round(5*T/delta_t)), mf_out1(1:round(5*T/delta_t)),
'LineWidth', 1);
hold on;
stem(t(idx_sample(1:5)), mf_out1(idx_sample(1:5)), 'r', 'filled');
xlabel('时间 (s)');
ylabel('幅度');
title('方法 1: y(t)*余弦载波匹配滤波输出');
legend('匹配滤波输出（不加噪声）','最佳采样时刻','Location','best');
box on; grid on;

subplot(2,1,2)
plot(t(1:round(5*T/delta_t)), mf_out2(1:round(5*T/delta_t)),
'LineWidth', 1);
hold on;
stem(t(idx_sample(1:5)), mf_out2(idx_sample(1:5)), 'r', 'filled');
xlabel('时间 (s)');
ylabel('幅度');
title('方法 2: y(t)先乘余弦后与方波匹配滤波输出');
legend('匹配滤波输出（不加噪声）','最佳采样时刻','Location','best');
box on; grid on;

%% 采样点偏移观察

offset_list = [0 1 2]; % 采样点偏离 0, 1, 2
for idx = 1:length(N0_list)
    n0 = N0_list(idx);
```

```
% AWGN 噪声
noise_power = n0/(2*delta_t);
noise = sqrt(noise_power) * randn(size(t));

% 接收信号
y_t = x_t + noise;

% 匹配滤波
mf1_out = conv_no_delay(y_t, fliplr(h1_t), delta_t);
y_base = y_t.*sqrt(2).*cos(2*pi*f_c*t);
mf2_out = conv_no_delay(y_base, fliplr(h2_t), delta_t);

% 遍历采样偏差
for j = 1:length(offset_list)
    offset_time = offset_list(j);

    idx_s = idx_sample + offset_time;

    new_ind = idx_s(1:length(idx_s)-1); %避免最后一个符号采样偏移溢出，统计 1~N-1 个符号
    x_hat1 = my_inverse_gray_map_real_M2(mf1_out(new_ind));
    x_hat2 = my_inverse_gray_map_real_M2(mf2_out(new_ind));

    bit_err1 = x_hat1 ~= bit_data(1:length(new_ind));
    bit_err2 = x_hat2 ~= bit_data(1:length(new_ind));

    BER_sim1(idx,j) = sum(bit_err1)/length(bit_err1);
    BER_sim2(idx,j) = sum(bit_err2)/length(bit_err2);
end

end

BER_theory = qfunc(sqrt(2*EsN0)); % BPSK 理论 BER
SER_theory = BER_theory; % 对 BPSK, SER=BER
SER_sim1 = BER_sim1;
SER_sim2 = BER_sim2;

%% ===== 绘制 BER/SER 曲线 =====
colors = lines(length(offset_list)); % 生成直观颜色
markers = {'o','s','d'}; % 不同偏差标记
figure; hold on;

for j = 1:length(offset_list)
```

```
    semilogy(EbN0_dB, BER_sim1(:,j), '-', 'Color', colors(j,:),  
'Marker', markers{j}, 'LineWidth',1.5);  
end  
  
for j = 1:length(offset_list)  
    semilogy(EbN0_dB, BER_sim2(:,j), '--', 'Color', colors(j,:),  
'Marker', markers{j}, 'LineWidth',1.5);  
end  
  
legend_entries = {};  
  
for j = 1:length(offset_list)  
    legend_entries{end+1} = sprintf('方法 1 offset %d 个采样点',  
offset_list(j));  
end  
for j = 1:length(offset_list)  
    legend_entries{end+1} = sprintf('方法 2 offset %d 个采样点',  
offset_list(j));  
end  
semilogy(EbN0_dB, BER_theory, '-', 'Color', 'r', 'LineWidth',1.5);  
legend_entries{end+1} = '理论值';  
set(gca, 'YScale', 'log');  
legend(legend_entries,'Location','best');  
xlabel('E_b/N_0 (dB)');  
ylabel('误码率 (BER)');  
title('BPSK 误比特率性能曲线');  
grid on; box on;  
  
figure;hold on;  
for j = 1:length(offset_list)  
    semilogy(EsN0_dB, SER_sim1(:,j), '-', 'Color', colors(j,:),  
'Marker', markers{j}, 'LineWidth',1.5);  
end  
  
for j = 1:length(offset_list)  
    semilogy(EsN0_dB, SER_sim2(:,j), '--', 'Color', colors(j,:),  
'Marker', markers{j}, 'LineWidth',1.5);  
end  
  
legend_entries = {};  
for j = 1:length(offset_list)  
    legend_entries{end+1} = sprintf('方法 1 offset %d 个采样点',  
offset_list(j));  
end
```

```
for j = 1:length(offset_list)
    legend_entries{end+1} = sprintf('方法 2 offset %d 个采样点',
offset_list(j));
end

semilogy(EsN0_dB, SER_theory, '-', 'Color', 'r', 'LineWidth',1.5);
legend_entries{end+1} = '理论值';

legend(legend_entries,'Location','best');
xlabel('E_s/N_0 (dB)');
ylabel('误比特率 (BER)');
title('BPSK 误码率性能曲线');
set(gca, 'YScale', 'log');
grid on; box on;

%% 随采样偏差变化的误码率
offset_list2 = -3:3;
BER_sim1_2 = zeros(size(offset_list2));
BER_sim2_2 = zeros(size(offset_list2));
n0 = N0_list(4);

% AWGN 噪声
noise_power = n0/2/delta_t;
noise = sqrt(noise_power) * randn(size(t));

% 接收信号
y_t = x_t + noise;

% 匹配滤波
mf1_out = conv_no_delay(y_t, fliplr(h1_t), delta_t);
y_base = y_t.*sqrt(2).*cos(2*pi*f_c*t);
mf2_out = conv_no_delay(y_base, fliplr(h2_t), delta_t);

% 遍历采样偏差
for j = 1:length(offset_list2)
    offset_time = offset_list2(j);

    idx_s = idx_sample + offset_time;

    new_ind = idx_s(1:length(idx_s)-1); %避免最后一个符号采样偏移溢出，统计 1~N-1 个符号
    x_hat1 = my_inverse_gray_map_real_M2(mf1_out(new_ind));
    x_hat2 = my_inverse_gray_map_real_M2(mf2_out(new_ind));
```



```
bit_err1 = x_hat1 ~= bit_data(1:length(new_ind));
bit_err2 = x_hat2 ~= bit_data(1:length(new_ind));

BER_sim1_2(j) = sum(bit_err1)/length(bit_err1);
BER_sim2_2(j) = sum(bit_err2)/length(bit_err2);
end

figure; hold on;
semilogy(offset_list2, BER_sim1_2, 'LineWidth',1.5);
semilogy(offset_list2, BER_sim2_2, 'LineWidth',1.5);
legend('方法 1','方法 2');
xlabel('采样偏差');
ylabel('误码率');
title('误码率随采样偏差变化');
set(gca, 'YScale', 'log');
grid on; box on;

%% 发送信号功率谱
% 设置滑动窗口参数
window_length = 1024; % 窗口长度
overlap = 512; % 重叠长度
nfft = 1024; % FFT 长度

% 初始化 PSD 估计
psdEstimate = zeros(nfft, 1);

% 滑动窗口处理
for start = 1:(window_length - overlap):(length(x_t) - window_length)
    % 提取窗口
    windowedSegment = x_t(start:start+window_length-1);

    % 窗函数处理（这里使用汉明窗）
    windowedSegment = windowedSegment .* hamming(window_length)';

    % 计算傅里叶变换
    Y = fft(windowedSegment, nfft);

    % 双边 = 单边 / 2
    Y = Y / 2;

    % 计算功率谱密度
    P2 = abs(Y/nfft).^2 ;

    % 将功率谱加入估计值（包括负频率）
```

```
    psdEstimate = psdEstimate + P2';
end

% 完成平均
psdEstimate = psdEstimate / ((length(x_t) - window_length) /
(window_length - overlap));

% 计算频率轴
f = (-nfft/2:nfft/2-1) / (nfft * delta_t);
%功率谱理论值
S_X = (A^2*T/2) * (sinc((f-f_c)*T).^2 + sinc((f+f_c)*T).^2);

%% 发送波形和接收波形
figure;
plot((delta_t:delta_t:10*T)', x0_t(1:round(10*T/delta_t)), 'LineWidth',
1);
hold on
plot((delta_t:delta_t:10*T)', x_t(1:round(10*T/delta_t)), 'LineWidth',
1);
xlabel('时间');
ylabel('波形');
title('前 10 个符号波形');
legend('基带脉冲成形信号','已调信号');
box on; grid on;

y_t = x_t + noise;
figure;
plot((delta_t:delta_t:10*T)', y_t(1:round(10*T/delta_t)), 'LineWidth',
1);
xlabel('时间');
ylabel('波形');
title('接收信号波形');
box on; grid on;

% 绘制功率谱
figure;
plot(f, (fftshift(psdEstimate)));
hold on
plot(f, (S_X));
xlabel('频率 (Hz)');
ylabel('功率谱密度 ');
title('Averaged Power Spectrum');

%%
```

```
% M=2, 由 bit 序列映射到电平符号
function mod_data = my_gray_map_real_M2(bit_data)
% 对应的符号序列长度
N = length(bit_data);
mod_data = zeros(1,N);
% 逐个符号判断映射关系
for n = 1:N
    current_bits = bit_data(n);
    if current_bits == 0
        mod_data(n) = -1;
    else
        mod_data(n) = 1;
    end
end
end

% M=2, 由电平符号映射到 bit 序列
function demod_bit_data = my_inverse_gray_map_real_M2(rx_data)
% 符号序列长度
N = length(rx_data);
demod_bit_data = zeros(1,N);
% 逐个符号判决
for n = 1:N
    current_level = rx_data(n);
    if current_level < 0
        demod_bit_data(n) = 0; % 负电平判决为 0
    else
        demod_bit_data(n) = 1; % 正电平判决为 1
    end
end
end
end

%时间对齐的卷积函数
function y_conv = conv_no_delay(y, h, delta_t)
% y: 输入信号
% h: 卷积核（匹配滤波模板）
% delta_t: 采样间隔
% y_conv: 卷积输出，长度与 y 相同，且对齐原始信号

% 确保行向量
y = y(:).';
h = h(:).';

Ny = length(y);
Nh = length(h);
```

```
% 初始化输出
y_conv = zeros(1, Ny);

% 直接计算卷积
for n = 1:Ny
    % 卷积窗口
    k_min = max(1, n-Nh+1);
    k_max = n;
    h_start = Nh - (k_max - k_min);
    h_end = Nh;

    y_conv(n) = sum(y(k_min:k_max) .* h(h_start:h_end)) * delta_t;
end
end
```

3. 实验数据记录及结果分析

(1) 给出前 10 个符号的发送波形和接收波形图。

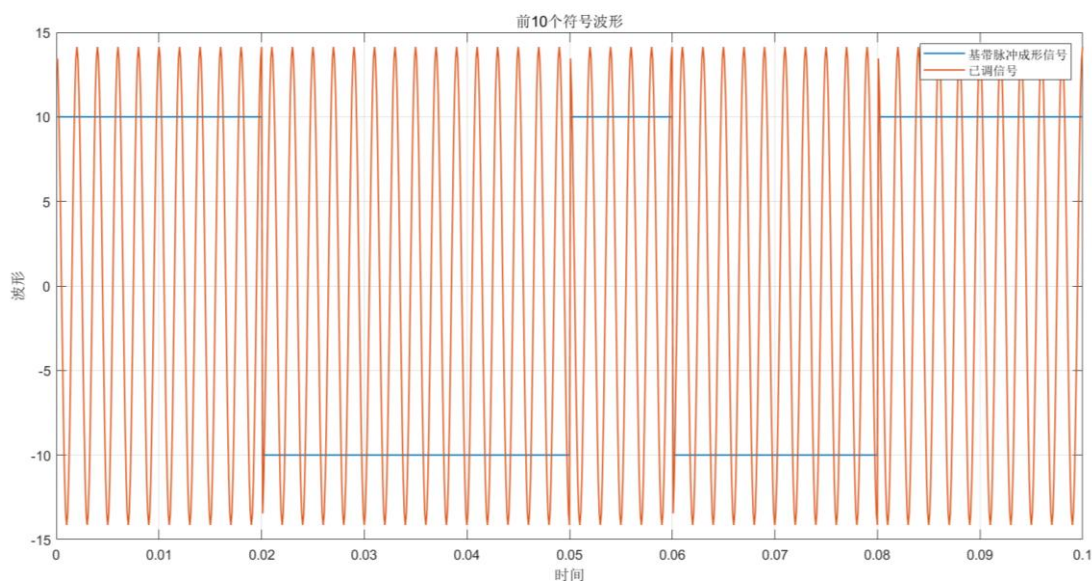


图 1 前 10 个符号的发送波形图

读图可知，前 10 个符号的基带信号为 $\{10, 10, -10, -10, -10, 10, -10, -10, 10, 10\}$ 。基带信号在时间轴上显示为持续 0.01 秒的+10 或-10 的矩形脉冲；已调信号在相同时间段内显示为 500Hz 的正弦波，但相位根据基带符号在 0° 和 180° 之间切换，所以在基带信号跃变处，已调信号也有突变。

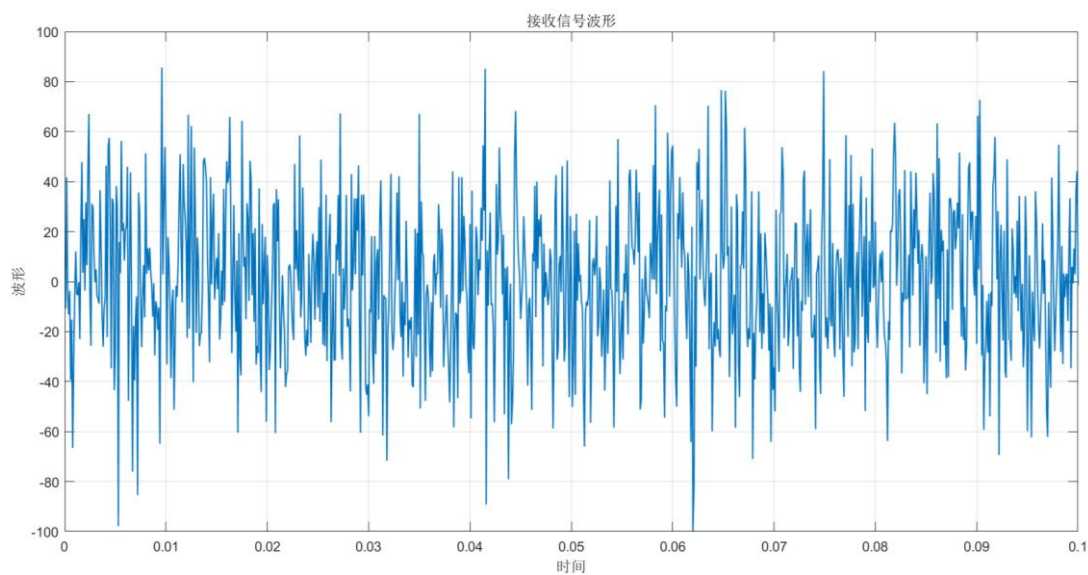


图 2 前 10 个符号的接收波形图

接收信号波形杂乱，无明显规律，主要原因是加入了 AWGN 噪声。

(2) 画出前 5 个符号的匹配滤波输出波形（采样前），并在图中明确标出最佳采样时刻。

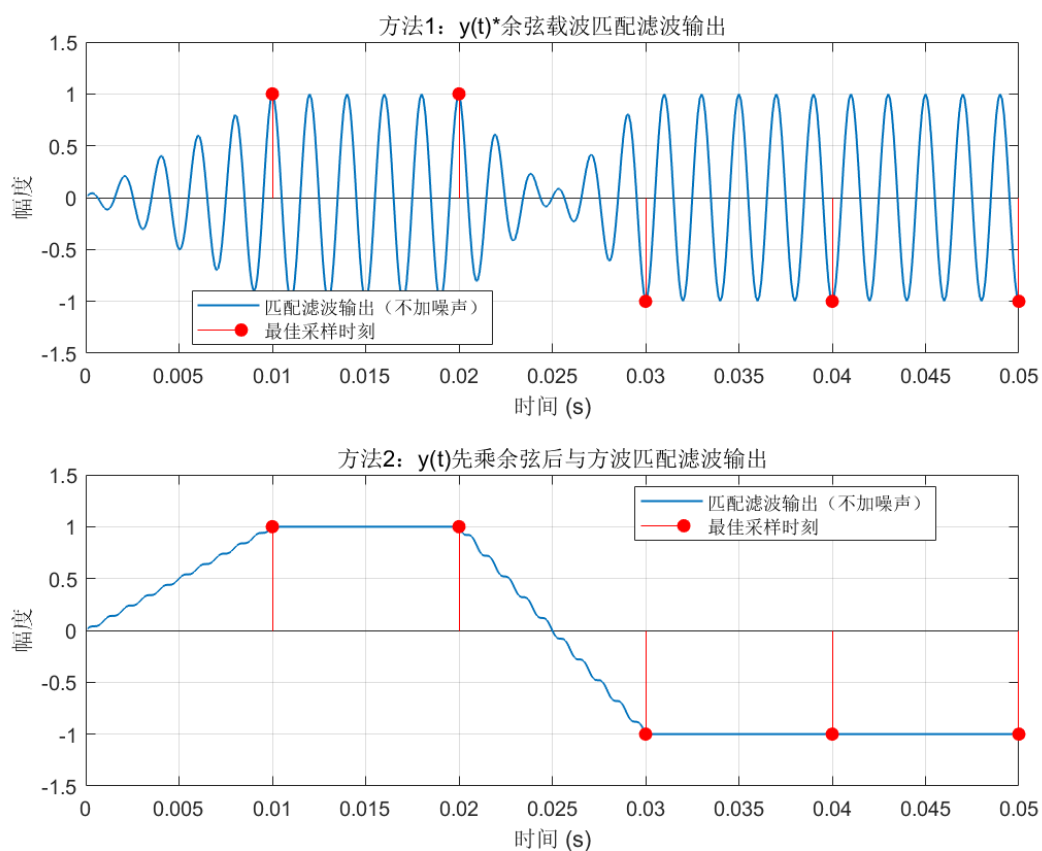


图 3 前 5 个符号的匹配滤波输出波形 2

由图可知，最佳采样时刻为 kT 。

在最佳采样时刻，两种方法采样到的匹配滤波输出均为幅度的峰值 ± 1 处，可以精确判决为 1 或 0，两种方法在最佳采样时刻的输出相同，说明这两种方法在 BPSK 调制情况下采样准确率均较高。

两种方法得到的匹配滤波输出图像不同，是因为选用了不同的匹配滤波方法。

(3) 分别比较误符号率和误比特率的理论值与仿真值。

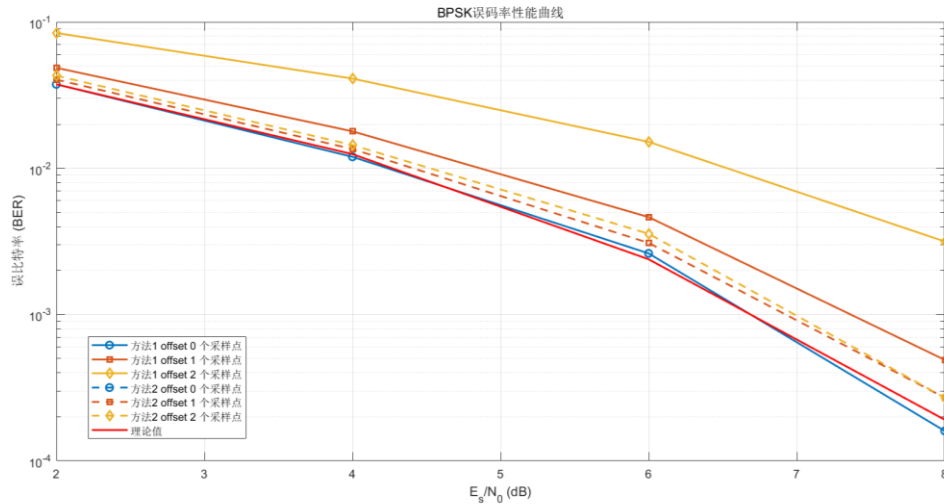


图 4 BPSK 误码率性能曲线

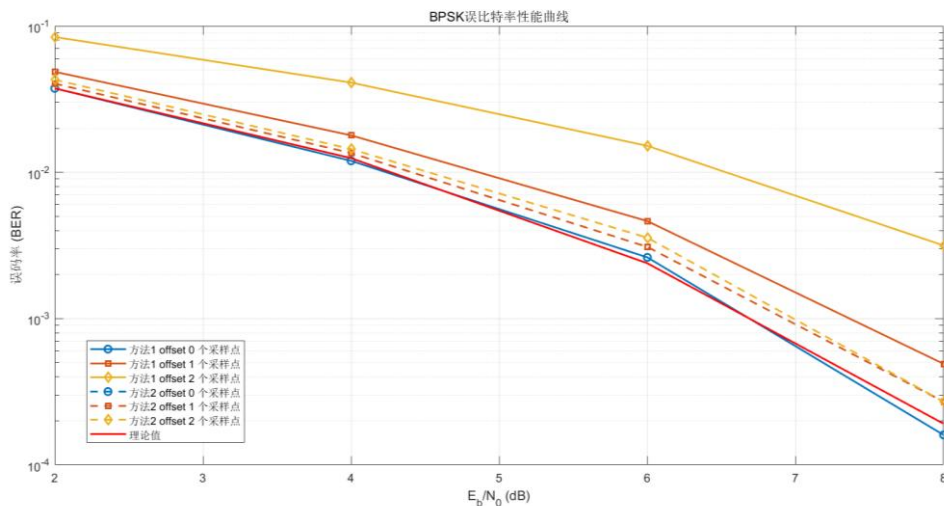


图 5 BPSK 误比特率性能曲线

BPSK 调制情况下，误码率与误比特率始终相等。由图可知，选用不同的匹配滤波方法、不同的采样点数量，误比特率/误码率的实验值与理论值接近，实验值整体略高于理论值，计算准确率较高。

随着 offset 的采样点数量增加，两种方法测得的误比特率/误码率与理论值的偏差逐渐

增大。横向比较两种匹配滤波方法，发现在 offset 相同采样点数量时，方法 2 的误比特率/误码率与理论值的偏差更小。

(4) 分析误码率随采样偏差变化的规律。

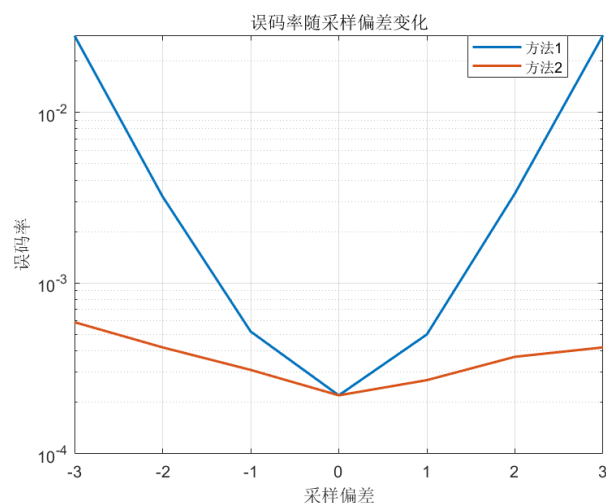


图 6 误码率随采样偏差变化

由图可知，方法 2 的误码率整体低于方法 1，采样偏差为 0 时两种方法的误码率相等。此外，随着采样偏差的绝对值越大，两种方法的误码率均增加，且图像对称。

(5) 比较画出的功率谱密度与理论功率谱密度。

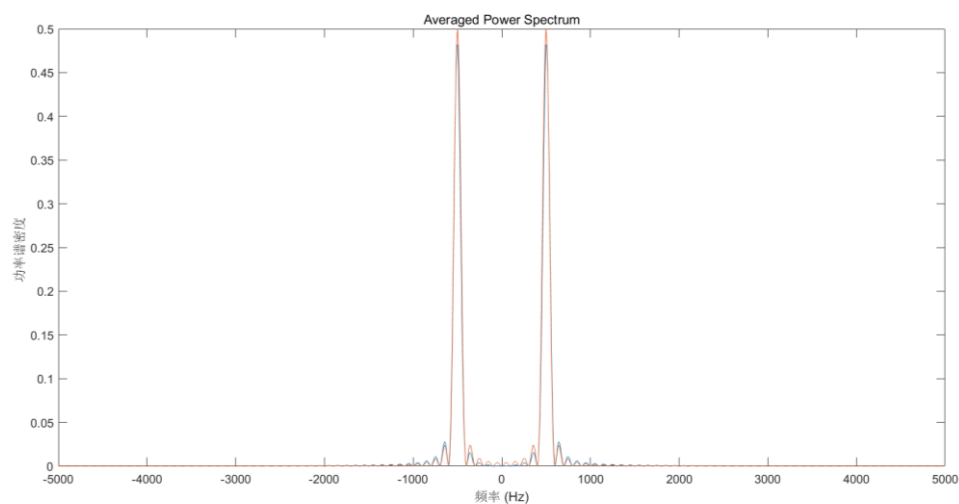


图 7 功率谱密度

由图可知，功率谱密度的实验值与理论值整体趋势和值接近，峰值位置上偏差最明显。

四、 4PAM 调制

1. 理论推导和计算

在 MPAM 调制下，误符号率和误码率分别为：

$$P_s = \frac{2M-2}{M} Q \left(\sqrt{\frac{3}{M^2-1} \cdot \frac{E_s}{n_0/2}} \right)$$

$$P_b = \frac{2M-2}{M \log_2 M} Q \left(\sqrt{\frac{3 \log_2 M}{M^2-1} \cdot \frac{E_b}{n_0/2}} \right)$$

MPAM 调制后，载波信号的功率谱密度为：

$$S_x(f) = \frac{A^2 T (M^2 - 1)}{6} \left[\left(\text{sinc}((f - f_0)T) \right)^2 + \left(\text{sinc}((f + f_0)T) \right)^2 \right]$$

代入 M=4，得：

$$\text{误符号率 } P_s = \frac{3}{2} Q \left(\sqrt{\frac{1}{5} \cdot \frac{E_s}{n_0/2}} \right)$$

$$\text{误比特率 } P_b = \frac{3}{4} Q \left(\sqrt{\frac{2}{5} \cdot \frac{E_b}{n_0/2}} \right)$$

$$\text{功率谱密度 } S_x(f) = \frac{5A^2 T}{2} \left[\left(\text{sinc}((f - f_0)T) \right)^2 + \left(\text{sinc}((f + f_0)T) \right)^2 \right]$$

2. 代码实现

```
% 通信与网络 实验 3 载波传输
%4PAM
clearvars;
close all;
clc;
rng(2023); %固定随机种子，保证结果可复现

M = 4;

% 重复实验次数，可修改
N = 1e5;

% 符号能量
Es = 1;

% 脉冲成形波形参数
T = 0.01; % 周期
A = sqrt(3*Es/(T*(M^2-1))); % 幅度
```



```
%载波参数
f_c=500; %载波频率

% 最小时间间隔
delta_t = 1e-4;

%仿真时间
t = (delta_t:delta_t:N*T);

%每个符号周期采样次数
num=T/delta_t;

% 成形脉冲
p_t = A * ones(num,1);

%初始化接收符号、判决符号
y=zeros(1,N);
x_hat=zeros(1,2*N);

%基带脉冲成形
x0_t=zeros(1,N*num);

%随机生成符号序列
bit_data = randi([0 1], 1, 2*N);
% 转化为调制后的电平序列 (mod_data)
mod_data = my_gray_map_real_M4(bit_data);

%脉冲成形
for cnt = 1:N
    x0_t(num*(cnt-1)+1:num*cnt) = mod_data(cnt)*p_t;
end

%4PAM 调制
x_t = x0_t.*sqrt(2).*cos(2*pi*f_c*t);

%% ===== 匹配滤波器 =====
h1_t = sqrt(2/T) * cos(2*pi*f_c*(0:delta_t:T-delta_t)); % 方法 1
h2_t = (1/sqrt(T)) * ones(1, num); % 方法 2
```

```
t_sample = T:T:N*T;
idx_sample = round(t_sample/delta_t);

%% ===== 仿真不同信噪比 =====
EbN0_dB = 2:2:8;          % 信噪比范围 dB
EbN0 = 10.^(EbN0_dB/10);
Eb = Es / log2(M);
N0_list = Eb ./ EbN0;
EsN0 = EbN0 * log2(M);
EsN0_dB = 10 * log10(EsN0);

ser_theory = zeros(size(EbN0));
ber_theory = zeros(size(EbN0));
%% 不加噪声观察匹配滤波
y_t = x_t;

% % 匹配滤波输出
%方法 1
mf_out1 = conv_no_delay(y_t, fliplr(h1_t), delta_t);

%方法 2
y_base = y_t.*sqrt(2).*cos(2*pi*f_c*t);
mf_out2 = conv_no_delay(y_base, fliplr(h2_t), delta_t);

% 前 5 个符号输出波形
figure;
subplot(2,1,1)
plot(t(1:round(5*T/delta_t)), mf_out1(1:round(5*T/delta_t)),
'LineWidth', 1);
hold on;
stem(t(idx_sample(1:5)), mf_out1(idx_sample(1:5)), 'r', 'filled');
xlabel('时间 (s)');
ylabel('幅度');
title('方法 1: y(t)*余弦载波匹配滤波输出');
legend('匹配滤波输出（不加噪声）','最佳采样时刻','Location','best');
box on; grid on;

subplot(2,1,2)
plot(t(1:round(5*T/delta_t)), mf_out2(1:round(5*T/delta_t)),
'LineWidth', 1);
hold on;
stem(t(idx_sample(1:5)), mf_out2(idx_sample(1:5)), 'r', 'filled');
xlabel('时间 (s)');
ylabel('幅度');
```

```
title('方法 2: y(t)先乘余弦后与方波匹配滤波输出');
legend('匹配滤波输出（不加噪声）','最佳采样时刻','Location','best');
box on; grid on;

%% 采样点偏移观察

offset_list = [0 1 2]; % 采样点偏离+-2, +-1, 0

BER_sim1 = zeros(length(EbN0_dB),length(offset_list)); % 方法 1 仿真 BER
BER_sim2 = zeros(length(EbN0_dB),length(offset_list)); % 方法 2 仿真 BER
SER_sim1 = zeros(length(EbN0_dB),length(offset_list)); % 方法 1 仿真 SER
SER_sim2 = zeros(length(EbN0_dB),length(offset_list)); % 方法 2 仿真 SER

for idx = 1:length(N0_list)
    n0 = N0_list(idx);

    % AWGN 噪声
    noise_power = n0/2/delta_t;
    noise = sqrt(noise_power) * randn(size(t));

    % 接收信号
    y_t = x_t + noise;

    % 匹配滤波
    %方法 1
    mf1_out = conv_no_delay(y_t, flip1r(h1_t), delta_t);

    %方法 2
    y_base = y_t.*sqrt(2).*cos(2*pi*f_c*t);
    mf2_out = conv_no_delay(y_base, flip1r(h2_t), delta_t);

    % 遍历采样偏差
    for j = 1:length(offset_list)
        offset_time = offset_list(j);

        idx_s = idx_sample + offset_time;

        new_ind = idx_s(1:length(idx_s)-1);

        y1 = mf1_out(new_ind);
        y2 = mf2_out(new_ind);

        x_hat1 = zeros(1,2*N-2);
        x_hat2 = zeros(1,2*N-2);
```

```
        for k = 1:N-1
            x_hat1((k-1)*2+1:k*2) =
my_inverse_gray_map_real_M4(y1(k)/(A*sqrt(T)));
            x_hat2((k-1)*2+1:k*2) =
my_inverse_gray_map_real_M4(y2(k)/(A*sqrt(T)));
        end

        % BER/SER 统计
        bit_err1 = xor(bit_data(1:2*N-2), x_hat1);
        bit_err2 = xor(bit_data(1:2*N-2), x_hat2);
        BER_sim1(idx,j) = sum(bit_err1)/length(bit_data);
        BER_sim2(idx,j) = sum(bit_err2)/length(bit_data);

        bit_err1_reshape = reshape(bit_err1,2,N-1);
        bit_err2_reshape = reshape(bit_err2,2,N-1);
        SER_sim1(idx,j) = sum(any(bit_err1_reshape,1))/(N-1);
        SER_sim2(idx,j) = sum(any(bit_err2_reshape,1))/(N-1);
    end
    ser_theory(idx) = (2*(M-1)/M) * qfunc(sqrt(6*EsN0(idx)/(M^2-1)));
    ber_theory(idx) = ser_theory(idx) / log2(M);
end

%% ===== 绘图 =====
colors = lines(length(offset_list)); % 生成直观颜色
markers = {'o','s','d'}; % 不同偏差标记
figure;
hold on;
set(gca, 'YScale', 'log');
for j = 1:length(offset_list)
    semilogy(EbN0_dB, BER_sim1(:,j), '-', 'Color', colors(j,:),
'Marker', markers{j}, 'LineWidth',1.5);
end

for j = 1:length(offset_list)
    semilogy(EbN0_dB, BER_sim2(:,j), '--', 'Color', colors(j,:),
'Marker', markers{j}, 'LineWidth',1.5);
end

legend_entries = {};

for j = 1:length(offset_list)
    legend_entries{end+1} = sprintf('方法 1 offset %d 个采样点',
offset_list(j));
end
```

```
for j = 1:length(offset_list)
    legend_entries{end+1} = sprintf('方法 2 offset %d 个采样点',
offset_list(j));
end
semilogy(EbN0_dB, ber_theory, '-', 'Color', 'r', 'LineWidth',1.5);
legend_entries{end+1} = '理论值';

legend(legend_entries,'Location','best');
xlabel('E_b/N_0 (dB)');
ylabel('误比特率 (BER)');
title('4PAM 误比特率性能曲线');
grid on; box on;

figure;hold on;
set(gca, 'YScale', 'log');
for j = 1:length(offset_list)
    semilogy(EsN0_dB, SER_sim1(:,j), '-', 'Color', colors(j,:),
'Marker', markers{j}, 'LineWidth',1.5);
end

for j = 1:length(offset_list)
    semilogy(EsN0_dB, SER_sim2(:,j), '--', 'Color', colors(j,:),
'Marker', markers{j}, 'LineWidth',1.5);
end

legend_entries = {};
for j = 1:length(offset_list)
    legend_entries{end+1} = sprintf('方法 1 offset %d 个采样点',
offset_list(j));
end
for j = 1:length(offset_list)
    legend_entries{end+1} = sprintf('方法 2 offset %d 个采样点',
offset_list(j));
end

semilogy(EsN0_dB, ser_theory, '-', 'Color', 'r', 'LineWidth',1.5);
legend_entries{end+1} = '理论值';

legend(legend_entries,'Location','best');
xlabel('E_s/N_0 (dB)');
ylabel('误码率 (BER)');
title('4PAM 误码率性能曲线');

grid on; box on;
```

```
%% 发送信号功率谱
% 设置滑动窗口参数
window_length = 1024; % 窗口长度
overlap = 512; % 重叠长度
nfft = 1024; % FFT 长度

% 初始化 PSD 估计
psdEstimate = zeros(nfft, 1); % Include both positive and negative
frequencies

% 滑动窗口处理
for start = 1:(window_length - overlap):(length(x_t) - window_length)
    % 提取窗口
    windowedSegment = x_t(start:start+window_length-1);

    % 窗函数处理（这里使用汉明窗）
    windowedSegment = windowedSegment .* hamming(window_length)';

    % 计算傅里叶变换
    Y = fft(windowedSegment, nfft);

    % 计算功率谱密度
    P2 = abs(Y/nfft).^2 / 4;

    % 将功率谱加入估计值（包括负频率）
    psdEstimate = psdEstimate + P2';
end

% 完成平均
psdEstimate = psdEstimate / ((length(x_t) - window_length) /
(window_length - overlap));

% 计算频率轴
f = (-nfft/2:nfft/2-1) / (nfft * delta_t);

%功率谱理论值
S_X = (A^2*T*(M^2-1)/6) * (sinc((f-f_c)*T).^2 + sinc((f+f_c)*T).^2);

%% 发送波形和接收波形
figure;
plot((delta_t:delta_t:5*T)', x0_t(1:round(5*T/delta_t)), 'LineWidth',
1);
hold on
```

```
plot((delta_t:delta_t:5*T)', x_t(1:round(5*T/delta_t)), 'LineWidth',
1);
xlabel('时间');
ylabel('波形');
title('前 5 个符号波形');
legend('基带脉冲成形信号','已调信号');
box on; grid on;

y_t = x_t + noise;
figure;
plot((delta_t:delta_t:5*T)', y_t(1:round(5*T/delta_t)), 'LineWidth',
1);
xlabel('时间');
ylabel('波形');
title('接收信号波形');
box on; grid on;

% 绘制功率谱
figure;
plot(f, (fftshift(psdEstimate)));
hold on
plot(f, (S_X));
xlabel('频率 (Hz)');
ylabel('功率谱密度 ');
title('功率谱密度');
%%

% M=4, 由 bit 序列映射到电平符号
function mod_data = my_gray_map_real_M4(bit_data)
if mod(length(bit_data),2)==0
    % 对应的符号序列长度
    N = length(bit_data)/2;
    mod_data = zeros(1,N);
    % 逐个符号判断映射关系
    for n = 1:N
        current_bits = bit_data(2*n-1 : 2*n);
        if current_bits(1)==0 && current_bits(2)==0
            mod_data(n) = -3;
        elseif current_bits(1)==0 && current_bits(2)==1
            mod_data(n) = -1;
        elseif current_bits(1)==1 && current_bits(2)==1
            mod_data(n) = 1;
        else
            mod_data(n) = 3;
        end
    end
end
```

```
        end
    end
else
    error('Invalid modulation order');
end
end

%M=4, 由电平符号映射到 bit 序列
function demod_bit_data = my_inverse_gray_map_real_M4(rx_data)
% 符号序列长度
N = length(rx_data);
demod_bit_data = zeros(1,2*N);
% 逐个符号判决
for n = 1:N
    current_level = rx_data(n);
    if current_level < -2
        demod_bit_data(2*n-1:2*n) = [0, 0]; % -3A -> 00
    elseif current_level < 0
        demod_bit_data(2*n-1:2*n) = [0, 1]; % -A -> 01
    elseif current_level < 2
        demod_bit_data(2*n-1:2*n) = [1, 1]; % A -> 11
    else
        demod_bit_data(2*n-1:2*n) = [1, 0]; % 3A -> 10
    end
end
end

function y_conv = conv_no_delay(y, h, delta_t)
% y: 输入信号
% h: 卷积核（匹配滤波模板）
% delta_t: 采样间隔
% y_conv: 卷积输出，长度与 y 相同，且对齐原始信号

% 确保行向量
y = y(:).';
h = h(:).';

Ny = length(y);
Nh = length(h);

% 初始化输出
y_conv = zeros(1, Ny);
```



```
% 直接计算卷积
for n = 1:Ny
    % 卷积窗口
    k_min = max(1, n-Nh+1);
    k_max = n;
    h_start = Nh - (k_max - k_min);
    h_end = Nh;

    y_conv(n) = sum(y(k_min:k_max) .* h(h_start:h_end)) * delta_t;
end
end
```

3. 实验数据记录及结果分析

(1) 给出前 5 个符号的发送波形和接收波形图。

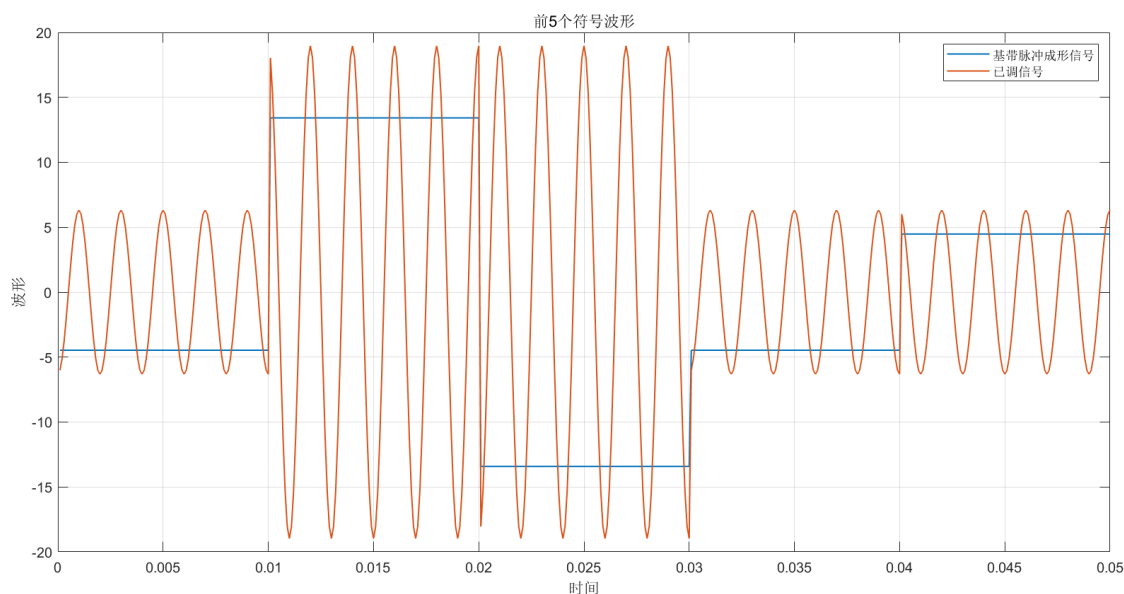


图 8 前 5 个符号的发送波形图

读图可知，前 5 个符号的基带信号大概率为 $\{-5, 15, -15, -5, 5\}$ ，因为加入了噪声所以存在一定偏差。基带信号在时间轴上显示为持续 0.01 秒的矩形脉冲；已调信号在相同时间段内显示为 500Hz 的正弦波，但相位和幅度根据基带信号会发生变化，所以在基带信号跃变处，已调信号也有突变。

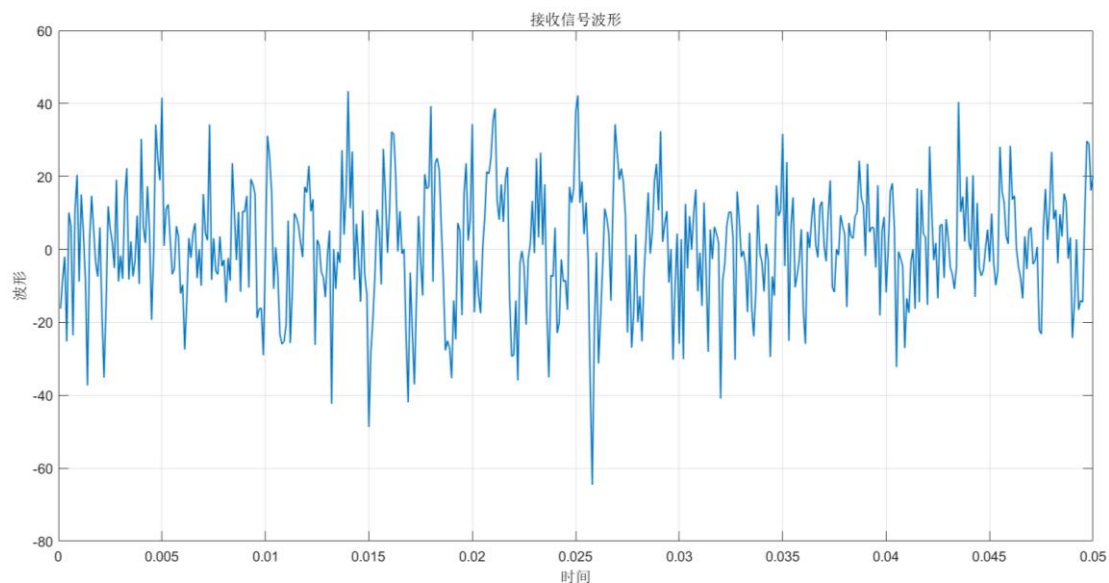


图 9 前 5 个符号的接收波形图

接收信号波形杂乱，无明显规律，主要原因是加入了 AWGN 噪声。

(2) 画出前 5 个符号的匹配滤波输出波形（采样前），并在图中明确标出最佳采样时刻。

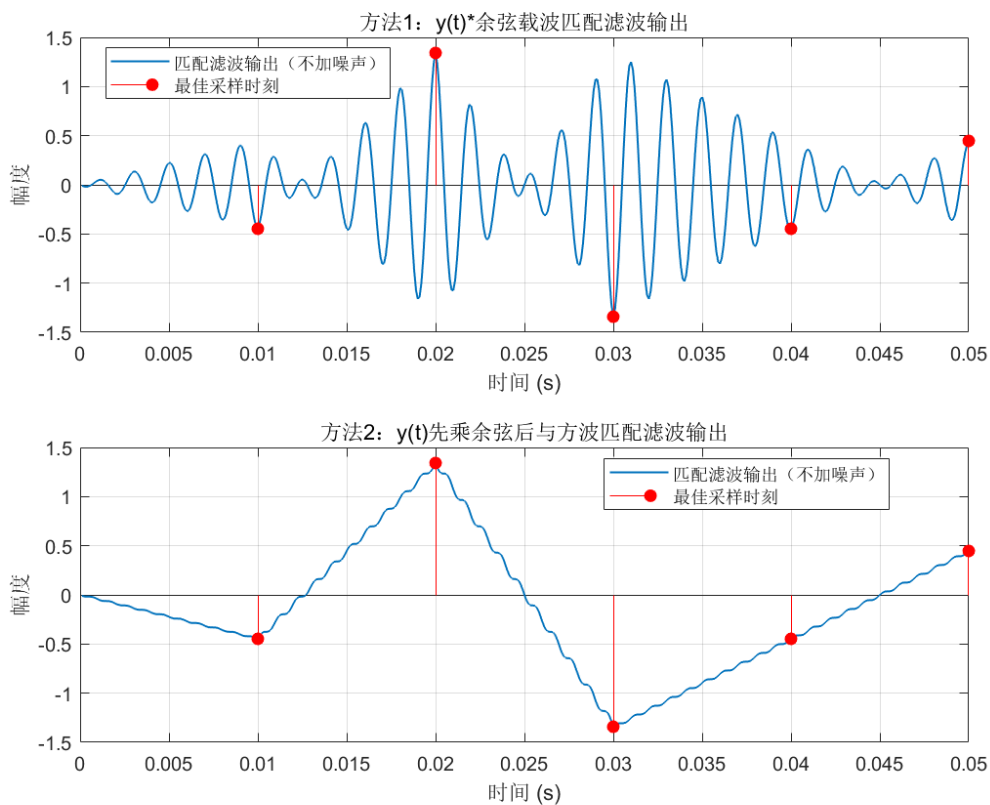


图 10 前 5 个符号的匹配滤波输出波形 2

由图可知，最佳采样时刻为 kT 。

在最佳采样时刻，两种方法采样到的匹配滤波输出均接近 ± 1.5 或 ± 0.5 幅度处，判决时会有误差但误差较小，两种方法在最佳采样时刻的输出相同，说明这两种方法在 MPAM 调制情况下采样准确率均较高。

两种方法得到的匹配滤波输出图像不同，是因为选用了不同的匹配滤波方法。

(3) 分别比较误符号率和误比特率的理论值与仿真值。

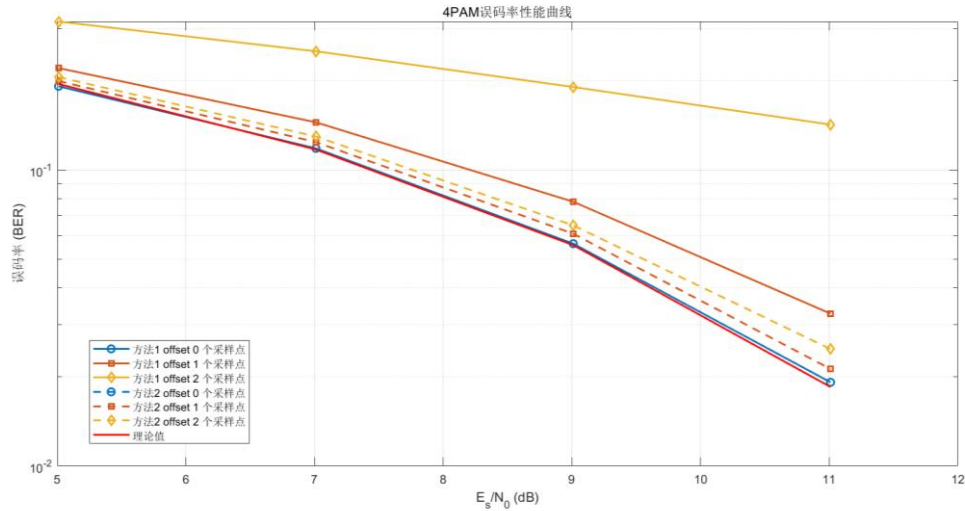


图 11 4PAM 误码率性能曲线

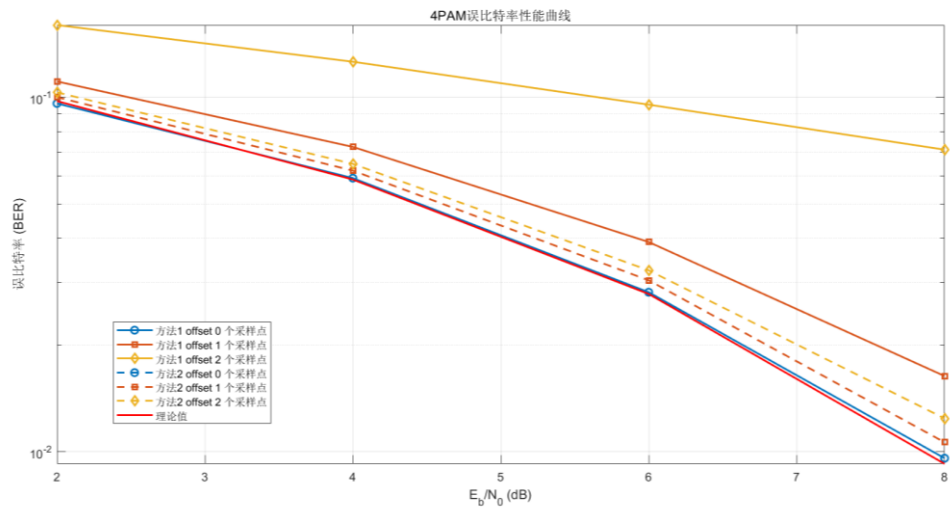


图 12 4PAM 误比特率性能曲线

4PAM 调制情况下，误码率与误比特率之间相差常数倍。由图可知，选用不同的匹配滤波方法、不同的采样点数量，误比特率/误码率的实验值与理论值较为接近，实验值整体略高于理论值，计算准确率较高。

随着 offset 的采样点数量增加，两种方法测得的误比特率/误码率与理论值的偏差逐渐增大，选用方法 1 且 offset 2 个采样点时实验值与理论值的误差较明显。横向比较两种匹配滤波方法，发现在 offset 相同采样点数量时，方法 2 的误比特率/误码率与理论值的偏差更小。

(4) 比较画出的功率谱密度与理论功率谱密度。

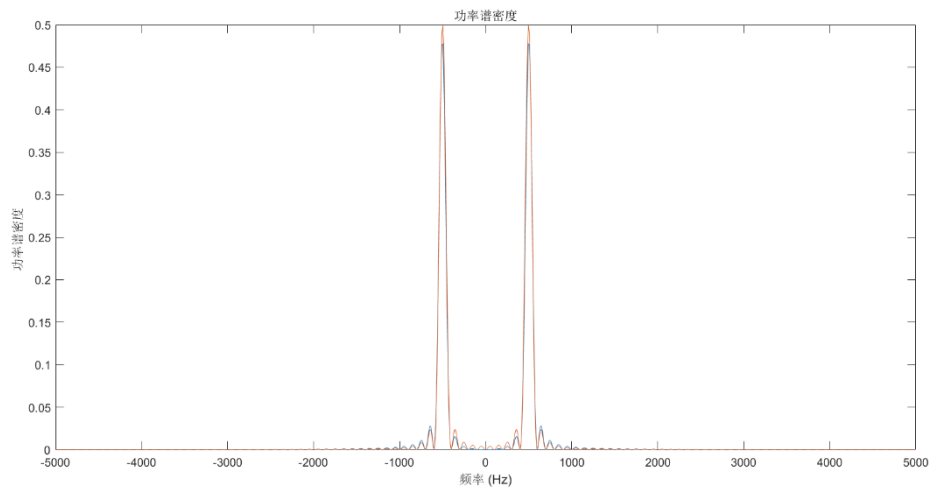


图 13 功率谱密度

由图可知，功率谱密度的实验值与理论值整体趋势和值接近，峰值位置上偏差最明显。

五、对比 BPSK 和 4PAM 的误码曲线

综合了两种载波传输的调制方法，参考前面的实验结果，选用了匹配滤波方法 2，offset 采样点数为 0 的情形，并绘制了 BPSK 和 4PAM 误比特率的性能对比图。由图可知，4PAM 调制时的误比特率整体高于 BPSK 调制。

观察 10^{-3} 误比特率所需的 E_b/n_0 ，发现实验测得两种方法相差 3.72dB，理论值相差 3.75dB。其原因在于 BPSK 的 $P_b = Q\left(\sqrt{\frac{E_b}{n_0/2}}\right)$ ，4PAM 的 $P_b = \frac{3}{4}Q\left(\sqrt{\frac{2}{5} \cdot \frac{E_b}{n_0/2}}\right)$ ，在相同 BER 的情况下， E_b/n_0 之间相差约 4dB。

BPSK理论值所需 E_b/N_0 : 6.69 dB
 4PAM理论值所需 E_b/N_0 : 10.44 dB
 理论值差距: 3.75 dB

BPSK仿真值所需 E_b/N_0 : 6.66 dB
 4PAM仿真值所需 E_b/N_0 : 10.38 dB
 仿真值差距: 3.72 dB

图 14 命令行输出

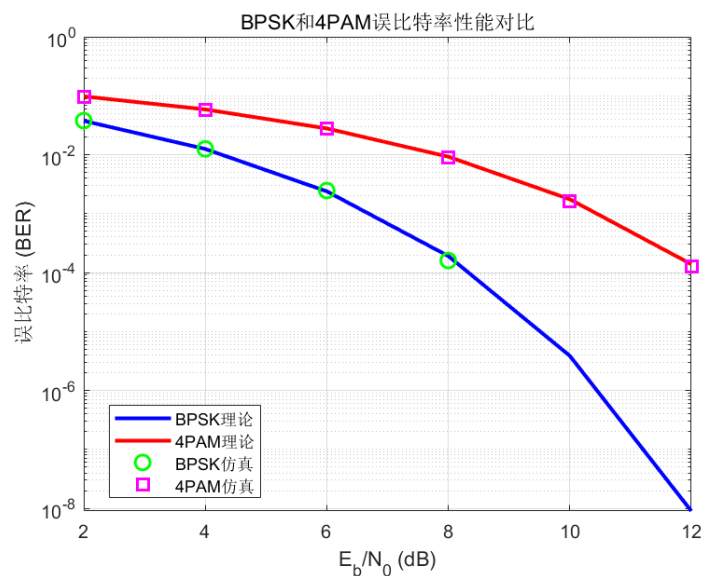


图 15 BPSK 和 4PAM 误比特率性能对比

六、实验总结

本次通信与网络课程实验通过 Matlab 对矩形包络下的 BPSK 与 4PAM 调制的载波传输进行仿真。实验需要对 BPSK 与 4PAM 调制的载波传输进行不同方法的匹配滤波，计算误符号率、误比特率、功率谱密度，并观察不同的匹配滤波方法、不同的 offset 采样点数、不同的载波调制方法对误符号率与误比特率的影响。通过本次实验，我基本熟悉了这两种调制方法及其相关应用，也能更熟练地结合理论公式调试代码。